RK3588 与 STM32 CAN 协议手册

版本：V1.0 日期：2026-07-16 适用下位机：STM32F407 线控底盘控制板 适用电机驱动：OID 运行模式：经典 CAN

1. 文档范围

本文档定义 RK3588 上位机与 STM32 下位机之间的 CAN 接口。RK3588 只应访问 STM32 的主控接口，不应直接发送 OID 电机驱动器报文。

本版本正式交付的接口如下：

方向	CAN ID	DLC	用途	周期
RK3588 -> STM32	`0x005`	8	底盘、割刀、升降、草袋控制	50 ms
STM32 -> RK3588	`0x507`	8	速度/角速度回显、模式、电量、温度和草袋状态	约 40 ms

以下接口不属于本次交付：

- `0x508`、`0x509`：当前 OID 配置下没有真实轮位置数据，暂不作为有效里程计接口。
- `0x506`：源码中存在旧的状态帧函数，但当前没有调用点。
- OID 节点 ID `0x001` 到 `0x004`：属于 STM32 内部电机总线协议。

2. CAN 总线参数

参数	取值
CAN 类型	经典 CAN，不使用 CAN FD
标识符	11 位标准帧
波特率	500 kbit/s
数据帧	标准数据帧
DLC	8
字节序	按字段说明；控制帧中的有符号单字节为二进制补码，反馈帧的多字节整数为大端序
校验	当前协议没有 CRC、序列号或应用层确认帧

STM32 源码实际使用的 CAN 时序为 APB1=42 MHz、Prescaler=6、BS1=9TQ、BS2=4TQ，对应 500 kbit/s。项目 `.ioc` 中的 875 kbit/s 记录已过期，RK3588 应配置为 500 kbit/s。

3. 控制帧：CAN ID `0x005`

3.1 帧格式

标准数据帧，DLC 必须为 8。RK3588 应每 50 ms 发送一次完整帧，即使控制量没有变化也不能停止发送。

Byte	字段	类型	缩放/范围	说明
0	`ctrl_mode`	`uint8`	`0`、`1`、`2`	API 状态机命令
1	`vx_raw`	`int8`	`vx = vx_raw * 0.1` m/s	前进速度；正值为前进，负值为后退
2	`vy_raw`	`int8`	当前忽略	差速底盘不支持侧向速度，固定填 `0`
3	`omega_raw`	`int8`	`omega = omega_raw * 0.1` rad/s	角速度；正负方向以现场坐标系标定为准
4	`blade_level`	`uint8`	`0` 到 `3`	割刀挡位
5	`lifter_level`	`uint8`	`1` 到 `5`	刀盘高度挡位；越界值会被钳位
6	`bag_target`	`uint8`	`0` 或非 `0`	`0` 请求关闭草袋，非 `0` 请求打开草袋
7	`magic_key`	`uint8`	`0xAA` 有效	显式清除碰撞锁存/系统锁；正常运行必须填 `0x00`

3.2 `ctrl_mode` 两步启动流程

`ctrl_mode=2` 不能直接启动。RK3588 必须按以下顺序操作：

值	名称	行为
`0`	退出 API	退出 API 模式；如果此前处于 API 模式，STM32 会停车并进入安全锁定，通常需要人工解锁
`1`	API 待机	挂载 API 控制权，但关闭底盘动力；同时清除上位机拦截锁
`2`	API 运行	只有当前状态为 `1` 且没有安全拦截锁时才生效，允许底盘执行速度指令

建议流程：

1. 发送 `ctrl_mode=1`，保持其他控制量为安全值。
2. 读取 `0x507`，确认 Byte4 为 `1`。
3. 发送 `ctrl_mode=2`，开始以 50 ms 周期发送控制帧。
4. 读取 `0x507`，确认 Byte4 为 `2` 后再发送非零速度。

如果发生超时、碰撞、急停或遥控器接管，STM32 会停止底盘并进入锁定状态。`magic_key=0xAA` 只能在上位机明确执行解锁操作时短暂使用，不能在每个 50 ms 帧中持续发送，否则可能反复清除安全锁存。

3.3 速度字段

源码按以下公式解析：

```
vx_mps      = int8(Byte1) * 0.1
omega_rad_s = int8(Byte3) * 0.1
vy_mps      = 0
```

当前工程注释约定的推荐范围为：

- `vx_raw`：`-10` 到 `10`，对应 `-1.0` 到 `1.0` m/s。
- `omega_raw`：`-10` 到 `10`，对应 `-1.0` 到 `1.0` rad/s。

`int8` 本身可以表达更大的数值，但当前协议没有独立的上位机范围错误反馈，RK3588 不应使用超出上述推荐范围的值。

3.4 割刀挡位 Byte4

值	PWM 占空比	含义
`0`	0%	关闭
`1`	45%	割刀 1 档
`2`	50%	割刀 2 档
`3`	55%	割刀 3 档

大于 `3` 的值会被下位机钳位为 `3`。割刀仍受下位机安全锁和故障逻辑约束。

3.5 刀盘高度挡位 Byte5

值	目标高度约值	含义
`1`	2 cm	最低档
`2`	5 cm	低档
`3`	7 cm	中档
`4`	9 cm	高档
`5`	11 cm	最高档

这是下位机根据时间积分和标定表估算的目标位置，不是 CAN 反馈的实测高度。小于 `1` 的值会被钳位为 `1`，大于 `5` 的值会被钳位为 `5`。

3.6 草袋目标状态 Byte6

值	目标
`0`	关闭草袋
非 `0`	打开草袋

该字段是目标状态，不是“翻转一次”的按键命令。STM32 会根据当前草袋状态决定是否启动打开或关闭动作；RK3588 每 50 ms 重复发送同一目标不会导致草袋反复翻转。

草袋开合动作由下位机状态机执行，单次动作超时约为 15 s。动作过程中 `0x507` 的草袋状态位可能仍保持上一次完成状态，动作完成后才更新为目标状态。

3.7 控制帧示例

以下示例均为 8 字节数据，最后一个字节正常填 `00`。

场景	CAN ID	DATA	说明
API 待机	`005`	`01 00 00 00 00 01 00 00`	`ctrl_mode=1`，速度为 0，刀盘关闭，升降 1 档，草袋关闭
API 运行	`005`	`02 00 00 00 00 01 00 00`	`ctrl_mode=2`，速度为 0
前进 0.5 m/s	`005`	`02 05 00 00 00 01 00 00`	`vx_raw=5`
前进 0.5 m/s、割刀 1 档、升降 3 档	`005`	`02 05 00 00 01 03 00 00`	API 运行中的典型控制帧
打开草袋	`005`	`02 00 00 00 00 03 01 00`	保持 API 运行，发送 50 ms 周期
关闭草袋	`005`	`02 00 00 00 00 03 00 00`	请求关闭草袋
API 内停车	`005`	`02 00 00 00 00 01 00 00`	保持 `ctrl_mode=2`，速度指令为 0
退出 API	`005`	`00 00 00 00 00 01 00 00`	退出 API，可能触发安全锁定

4. 状态反馈帧：CAN ID `0x507`

4.1 帧格式

标准数据帧，DLC=8。STM32 传感器任务按约 40 ms 周期发送。Byte0 到 Byte3 在 API 模式下是上位机速度指令的回显，不是 OID 电机的实测速度。

Byte	字段	类型/编码	说明
0-1	`vx_feedback`	`int16` 大端，数值 / 1000	API 模式下为 `vx` 指令回显，单位 m/s
2-3	`omega_feedback`	`int16` 大端，数值 / 1000	API 模式下为 `omega` 指令回显，单位 rad/s
4	`ctrl_mode`	`uint8`	当前 API 状态：0、1、2
5	`battery_soc`	`uint8`	BMS RSOC，正常范围 0 到 100，单位 %
6	`bms_temperature`	`uint8`	BMS 温度，单位 ℃；`0xFF` 表示未知/异常
7	`status_flags`	`uint8`	草袋、割刀状态位，见下表

4.2 `status_flags` 位定义

Bit	掩码	含义	取值
0	`0x01`	草袋实际开合状态	1=已打开，0=已关闭；动作中可能保持旧值
1	`0x02`	草袋连接状态	1=已连接，0=未连接
2	`0x04`	草袋满载状态	1=满载，0=未满
3	`0x08`	保留	置 0
4-5	`0x30`	割刀挡位	`0` 到 `3`，右移 4 位取得
6-7	`0xC0`	保留	置 0

4.3 反馈示例

```
CAN ID = 507
DATA   = 01 F4 FE D4 02 50 23 13
```

解析结果：

- `01 F4` = 500，`vx_feedback=0.500 m/s`。
- `FE D4` = -300，`omega_feedback=-0.300 rad/s`。
- Byte4=`02`，当前为 API 运行。
- Byte5=`50`，电量 80%。
- Byte6=`23`，BMS 温度 35 ℃。

- Byte7=`13`，二进制为 `0001 0011`：草袋已打开、已连接、割刀 1 档，未满载。

4.4 已知限制

- 当前反馈没有命令应答、序列号、锁定原因、故障码或升降实际高度字段。
- Byte5 在源码中沿用变量名 `voltage_int`，但实际写入的是 BMS RSOC。BMS 无有效数据且缓存超过约 3 s 时，当前实现可能写入 `0x01` 作为故障占位；这与 1% 电量数值存在歧义。
- 当前配置不提供真实轮速/里程计反馈，RK3588 不应把 `0x508`、`0x509` 当作有效位置数据。

5. RK3588 推荐使用流程

5.1 初始化

1. 配置 CAN 为 500 kbit/s、11 位标准帧、经典 CAN。
2. 接收过滤至少包含 `0x507`。
3. 启动后先发送 `0x005` 的 API 待机帧，不要直接发送 `ctrl_mode=2`。

5.2 运行

1. 发送 `ctrl_mode=1`，确认收到的 `0x507` Byte4 为 `1`。
2. 发送 `ctrl_mode=2`，确认收到的 `0x507` Byte4 为 `2`。
3. 之后每 50 ms 发送完整的 `0x005` 控制帧。
4. 正常运行时 Byte7 固定为 `0x00`。
5. 解析 `0x507` 的 Byte4 和 Byte7；不要把 Byte0-3 当作 OID 实测速度。

5.3 停车和退出

- 仅停车但继续保持 API：保持 Byte0=`2`，将 Byte1 和 Byte3 置 0。
- 关闭草袋：在 API 运行期间持续发送 Byte6=`0`，等待动作完成后再退出 API。
- 退出 API：发送 Byte0=`0`。当前安全逻辑可能锁定底盘，后续重新接管时应按项目的解锁流程处理。
- 控制帧连续超过 500 ms 未到达时，STM32 会停车并进入安全锁定；RK3588 必须使用 50 ms 周期发送，不能只在数值变化时发送。

6. SocketCAN 联调示例

以下命令假定 RK3588 的 CAN 接口名为 `can0`，实际接口名和权限以设备驱动为准。

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 500000
sudo ip link set can0 up

# 监听 STM32 状态反馈
candump can0,507:7FF

# API 待机
cansend can0 005#0100000000010000

# API 运行
cansend can0 005#0200000000010000

# API 运行，前进 0.5 m/s，割刀 1 档，升降 3 档，草袋关闭
cansend can0 005#0205000001030000

# 打开草袋；应继续按 50 ms 周期发送同一目标帧
cansend can0 005#0200000000030100
```

`cansend` 的 CAN ID 不带 `0x` 前缀，数据区不写空格。使用 `candump` 时应确认收到的 ID 是 `507`，且 Byte4 能按状态机变化为 `1` 或 `2`。

7. 版本记录

版本	日期	说明
V1.0	2026-07-16	根据 STM32 当前 OID 配置源码整理 RK3588 主控协议；确定 500 kbit/s、50 ms 控制周期、两步 API 启动；补齐 Byte6 草袋目标状态执行逻辑。